

Engineering Take-Home Test

Build a containerised RAG-based chat application over a Wikipedia article, using a local LLM and an AI-agent driven development workflow.

Role	Senior / Mid-Level Engineer (Full-Stack / AI)
Estimated Time	8–12 hours of focused work, spread over up to 5 calendar days
Submission Deadline	3 calendar days from the date this document was sent to you
Submission Format	Public Git repository link (GitHub / GitLab / Bitbucket)
Evaluation	Code quality, AI-assisted workflow, planning artefacts, test coverage, and a working containerised app

1. Overview

This take-home is designed to evaluate two things in equal measure: how you build software, and how you collaborate with AI agents while doing it.

The product itself is small and scoped — a web app that ingests a Wikipedia article and lets a user chat with its contents using a local LLM. What we genuinely care about is how you plan, decompose, and execute the work using AI tooling, and whether your codebase reflects that planning.

If you treat this as a one-shot prompt to an AI and ship whatever it produces, it will show. If you treat it as an engineering exercise where AI is your pair, that will also show. We strongly prefer the second.

2. The Problem

Build a small web application with the following user flow:

1. The user pastes a Wikipedia article URL into a form.
2. The app scrapes the article content (text body, sections, references where reasonable).
3. The app generates a concise summary of the article using a local LLM and displays it to the user.
4. The app chunks the article, generates embeddings, and stores them in a vector database.

5. Below the summary, a chat box lets the user ask questions about the article. Answers must be grounded in the retrieved chunks (RAG), not in the model's general knowledge.

That is the entire scope. Resist the urge to add auth, multi-article history, user accounts, or analytics. We are evaluating depth on a small surface, not breadth.

3. Hard Requirements

These are non-negotiable. Submissions that miss any of them will not be reviewed in detail.

3.1 AI-Agent Driven Development

- The application must be built with the help of AI agents — Cursor, Claude Code, Cline, Aider, Continue, or similar. Plain copy-pasting from a chat UI is not sufficient.
- You must use the agent for planning and decomposition, not only for code generation. We want to see the plan before the code.
- Free-tier options are fine. You may use the Cursor free tier, or sign up at build.nvidia.com to get free-tier access to models that can be plugged into your IDE.

3.2 Required Planning Artefacts

The repository root must contain the following three Markdown files. They are the primary artefact of your AI-driven workflow and we will read them carefully.

REQUIREMENTS.md — your interpretation of this brief. Functional and non-functional requirements, in-scope vs out-of-scope, assumptions you made, and open questions you decided unilaterally (and why).

DESIGN.md — the architecture. Components, data flow from URL to answered question, choice of LLM runtime, chunking strategy, embedding model, vector store, and the contracts between modules. Include a simple diagram (ASCII or image).

TASKS.md — the execution log. The decomposed task list you worked from, in the order you tackled them, with status. This should read like a real plan that an agent could pick up and execute, with notes on what you delegated to the AI and what you wrote by hand.

These three files together should make it possible for a new engineer to understand both what you built and how you arrived at it.

3.3 Local LLM

- Summarisation and chat must run against a locally-deployed LLM via Ollama or vLLM. No calls to OpenAI, Anthropic, Gemini, or other hosted inference APIs in the running application.
- Embeddings may use a local model (preferred) or a small hosted embedding model — call out the choice in DESIGN.md.

- During development, you may use AI agents that call hosted models — that is the workflow we are asking for. The constraint applies to the running application, not to your editor.

3.4 Vector Database

- Any vector DB is acceptable: Qdrant, Chroma, Weaviate, Milvus, pgvector, FAISS, or similar.
- It must run as part of the containerised stack, not as an in-process toy unless your justification in DESIGN.md is convincing.

3.5 Containerisation

- The full stack must come up with a single command — typically docker compose up.
- This includes the web app, the vector DB, and the LLM runtime (or clear, documented instructions for connecting to a host-side Ollama if model size makes containerising impractical).
- No secrets committed. Use .env.example for any required configuration.

3.6 Test Coverage

- Minimum 85% line coverage on the application code. Generate a coverage report and commit it (or a screenshot) to the repo.
- Tests must be meaningful. We will sample them. Tests that exist only to inflate coverage will count against you, not for you.
- Mock the LLM and vector DB at the unit-test layer; have at least one integration test that exercises the real wired-up stack.

4. Tech Choices

Pick the stack you are most productive in. The brief is language-agnostic. Common reasonable picks:

- Backend: NodeJs.
- Frontend: a minimal React / Next.js / Svelte app, or even server-rendered HTML with HTMX. We are not grading visual design.
- LLM runtime: Ollama with a small model such as llama3.2:3b, qwen2.5:3b, or phi3:mini works well on a developer laptop.
- Vector DB: Qdrant or Chroma are the easiest to get running in compose.

Justify your choices briefly in DESIGN.md. We are not looking for the trendiest stack — we are looking for one you can defend.

5. Suggested Workflow With AI Agents

This is a suggestion, not a script. Adapt it to how you actually work.

6. Start in your AI IDE (Cursor or similar). Create REQUIREMENTS.md by walking the agent through this brief and producing your own restatement of it. Resolve ambiguities explicitly.
7. Generate the first draft of DESIGN.md collaboratively. Push back on the agent. If it suggests something you would not ship, ask why. Capture the trade-offs in the doc.
8. Have the agent decompose the work into TASKS.md — small, sequenced, individually testable tasks. Aim for tasks small enough that a single agent run can complete one.
9. Execute the tasks one by one. Commit after each meaningful task. Update TASKS.md as you go so the document and the repo stay in sync.
10. Write tests alongside features, not at the end. Coverage gaps caught late are painful and obvious in review.
11. Containerise once the happy path works end-to-end on bare metal, not before.

If you use build.nvidia.com to access free-tier models from your IDE, note the model and integration approach in DESIGN.md.

6. Submission

Submit a link to a public Git repository. The repository must contain:

- REQUIREMENTS.md, DESIGN.md, TASKS.md at the root.
- README.md with run instructions, prerequisites, and any caveats. We should be able to clone and run with a single command.
- docker-compose.yml (or equivalent) wiring the full stack.
- Source code with tests and a coverage report.
- A short screen recording (2–4 minutes) or a few screenshots showing the app working end-to-end. Loom, a GIF, or committed PNGs are all fine.

Optionally, include a brief NOTES.md describing what you would change with another two days, and what your AI agent got wrong that you had to correct.

7. Evaluation Rubric

We score across six areas. The weights below are indicative, not strict.

Area	Weight	What we look for
AI-Assisted Workflow	25%	Quality of DESIGN.md, REQUIREMENTS.md, TASKS.md.

		Evidence that AI agents were used to plan, scaffold, and iterate, not just autocomplete.
Functionality	20%	End-to-end flow works: scrape → summarise → embed → chat. Sensible error handling for bad URLs and empty articles.
Code Quality & Architecture	20%	Clear separation of concerns, readable code, sensible abstractions for the LLM and vector store so they can be swapped.
Testing	15%	≥85% line coverage with meaningful tests (not assertion padding). Coverage report included in the repo.
Containerisation & DX	10%	Single command brings the stack up. Reasonable image sizes, no secrets in the repo, clean docker-compose.
Documentation & Clarity	10%	README explains how to run it. Trade-offs are stated. Known limitations are honest.

8. Ground Rules

- Be honest about what the AI wrote and what you wrote. We are not penalising AI-generated code — we are evaluating your judgement in directing it. Lying about authorship is the only thing here that is disqualifying.
- Do not commit API keys, even revoked ones. Use .env.example.
- If you hit a hard blocker — model won't run on your machine, a library is broken, etc. — document the workaround in NOTES.md and move on. Shipping something honest beats shipping nothing.
- If you finish early, do not pad. We would rather see a tight, well-tested 60% solution than a sprawling 100% one.

9. FAQ

Can I use a hosted LLM during development to save time?

Yes, through your AI IDE. The running application — what we will execute when reviewing — must use a local LLM via Ollama or vLLM.

My laptop cannot run a 7B model. What do I do?

Use a smaller model (3B class). Document the choice and its impact on quality in DESIGN.md. We would rather see a working app on a small model than a stalled one on a large model.

How polished does the UI need to be?

Functional and unembarrassing. A single page with a URL input, a summary, and a chat box is enough. We are not grading aesthetics.

Can I add features beyond the brief?

Only after the core flow is solid and tested. Scope creep before the basics are clean is a red flag, not a green one.

How strict is the 85% coverage threshold?

Strict, because hitting it forces you to write testable code. We will, however, ignore generated boilerplate, framework glue, and the entrypoint file when calculating it — call out any exclusions in your coverage config.

10. A Final Note

We designed this exercise to look like the work we actually do. Small, well-defined problem. Real constraints. AI agents in the loop. The candidates who do best are not the ones who produce the most code — they are the ones whose REQUIREMENTS.md, DESIGN.md, and TASKS.md make the code feel inevitable.

Good luck. We are looking forward to reading your repo.